

OpenMimicry

Project Design Document

A modular avatar interface layer for LLMs, voice, vision, and external agentic task runtimes.

Version 1.0

Author: Gabriel Henrique

Date: June 30, 2026

License: MIT

`github.com/<gabriel>/openmimicry`

Contents

Each chapter is independently readable. Cross references between chapters use chapter names, so jumping straight to a topic that interests you should not require reading the chapters before it.

Contents	2
1. Executive summary	7
1.1 At a glance	7
1.2 Five things you can do with OpenMimicry	8
2. Goals and non-goals	9
2.1 In scope	9
2.2 Explicitly out of scope (for now)	9
3. Architectural philosophy	11
3.1 Hexagonal core	11
3.2 Events, not RPC	11
3.3 One source of truth per concern	11
3.4 Honest seams	11
4. Layered architecture	12
4.1 The five layers	12
4.2 Visual: the layered diagram	12
4.3 Rules every layer obeys	13
4.4 Conceptual layers vs installable packages	13
5. Repository layout	14
5.1 Why this shape	14
6. Foundation layer (openmimicry-core)	16
6.1 What lives in core	16
6.2 What does NOT live in core	16
6.3 The Protocols	16
6.4 The schemas	17
6.5 The EventBus	18
6.6 Runtime	18
6.7 The contract-freeze rule	18
7. Sensor layer	19
7.1 openmimicry-stt (facade)	19

Contents	19
How the STT runs	19
7.2 openmimicry-vision (M13)	20
Detectors	20
Classifiers	20
Pipeline primitives	20
Privacy posture (mandatory)	20
Entry-point registries	21
8. Cognition layer	22
8.1 Components	22
8.2 openmimicry-llm	22
Adding a non-LiteLLM provider	23
8.3 AvatarDirector	23
8.4 detect_task_intent	23
8.5 TaskRouter	24
9. Effector layer	25
9.1 openmimicry-tts (facade)	25
Contents	25
9.2 openmimicry-avatar	25
The five modalities	25
AvatarRuntimeAdapter (the contract)	26
AvatarOrchestrator.swap_runtime invariant	26
9.3 openmimicry-tasks	26
Concrete adapters	26
Security posture for LocalShell	27
10. Surface layer	28
10.1 The wiring.py rule	28
10.2 apps/backend	28
10.3 apps/desktop/frontend	28
10.4 apps/desktop/src-tauri	29
10.5 apps/unity-bridge	29
10.6 apps/external-echo	29
11. Event flows	30
11.1 Text input	30
11.2 Push-to-talk	30
11.3 Wake-name live mode	30

11.4 Task delegation	30
11.5 Vision gesture.....	31
12. Configuration system	32
12.1 Profiles shipped at v1.0	32
12.2 Example app.yaml	32
12.3 The doctor script.....	33
13. Avatar modalities in depth	34
13.1 Sprite2D (default)	34
Pack layout	34
pack.yaml.....	34
13.2 Three.js (M9)	35
13.3 Live3D (M10).....	35
13.4 Unity (M11).....	35
13.5 External (M12).....	35
14. Voice modes	36
14.1 Text-always-on	36
14.2 Push-to-talk (PTT)	36
14.3 Live wake	36
14.4 Agent voice on/off.....	36
14.5 Barge-in (interruption).....	36
14.6 Echo handling.....	36
15. Task delegation in depth	37
15.1 The contract	37
15.2 LocalShellAdapter — the safe baseline	37
15.3 ClaudeCodeAdapter	37
15.4 MCPAgentAdapter.....	37
15.5 TaskRouter.....	37
16. Vision (M13) in depth	39
16.1 Pipeline.....	39
16.2 Gesture rules (zero-training).....	39
16.3 Movement rules (temporal).....	39
16.4 Plug-in registries.....	39
16.5 Why MediaPipe	40
17. Privacy and security posture	41
17.1 What stays on the machine by default	41
17.2 What does leave the machine — only when configured.....	41

17.3 Mandatory consent gate for vision	41
17.4 LocalShellAdapter safety	41
17.5 Disclosed third-party calls	42
17.6 Secrets management.....	42
18. Testing, CI, and release	43
18.1 The four test layers	43
18.2 Coverage	43
18.3 Linting and typing	43
18.4 Frontend tests.....	43
18.5 Tauri tests.....	43
18.6 GitHub Actions workflows	44
18.7 Versioning	44
19. Engineering practices.....	45
19.1 Tooling baseline	45
19.2 Conventional commits and changelog	45
19.3 Pre-commit hooks (optional)	45
19.4 Contributor documents	45
19.5 Docker	46
20. Running from zero.....	47
20.1 Prerequisites	47
20.2 Three workflows	47
Browser dev (no Tauri).....	47
Native Tauri (recommended for everyday use)	47
Docker (backend only)	48
20.3 Configuring an LLM	48
21. Milestones (M0 - M13)	49
21.1 Phase 0 — Contract freeze.....	49
21.2 MX — Tooling baseline.....	49
21.3 M0 — Core runtime	49
21.4 M1 — openmimicry-llm	49
21.5 M2 — openmimicry-voice	49
21.6 M3 — openmimicry-avatar core	49
21.7 M4 — Sprite2D.....	49
21.8 M5 — openmimicry-tasks	49
21.9 M6 — apps/backend.....	50
21.10 M7 — apps/desktop/frontend.....	50

21.11 M8 — apps/desktop/src-tauri	50
21.12 M9 — Three.js modality	50
21.13 M10 — Live3D	50
21.14 M11 — Unity bridge	50
21.15 M12 — External	50
21.16 M13 — openmimicry-vision	50
22. Roadmap (post-v1.0)	51
22.1 Cadence	51
23. FAQ	52
23.1 Why is it called OpenMimicry?	52
23.2 Why both openmimicry-voice and openmimicry-stt/-tts?	52
23.3 Can I use OpenMimicry without an LLM key?	52
23.4 Why Tauri instead of Electron?	52
23.5 Why MediaPipe instead of TensorFlow.js or PyTorch?	52
23.6 Why an event bus and not just function calls?	52
23.7 Why one wiring.py and not dependency injection magic?	53
24. Glossary	54
25. Appendix A — Make targets and Windows wrappers	56
26. Appendix B — Entry-point groups	57
27. Closing notes	58

1. Executive summary

OpenMimicry is an open-source desktop companion framework that connects animated 2D and 3D avatars to large language models, voice systems, computer vision, and external agentic task runtimes. Its purpose is to be a portfolio-grade, local-first, lightweight Jarvis-style companion that runs on Windows and Linux, sits in a transparent desktop overlay, can listen and speak, and can delegate work to systems such as Claude Code, MCP-based agents, or local shell allowlists, all through one consistent contract-driven Python core.

The project is delivered as a single Git repository organised as a workspace of independently installable Python packages, a FastAPI backend, a React and Vite frontend, a Tauri 2 desktop shell, an optional Unity bridge, a reference External avatar runner, and a set of bundled character packs. Every cross-module boundary is a frozen Pydantic contract or a runtime-checkable Python Protocol declared in openmimicry-core. Every concrete adapter (LLM provider, STT engine, TTS engine, avatar runtime, vision detector, task runtime) plugs in through that contract surface and is selected at runtime by configuration.

At version 1.0 the framework ships with end-to-end working implementations of all five canonical modules described in the original project brief: a core runtime, an avatar layer with five rendering modalities, a voice layer, an LLM layer, and a task-delegation layer. It also ships a sixth, vision module (M13) that adds gesture and movement detection through MediaPipe, behind a strict consent-gated, off-by-default privacy posture.

This document is the full design reference for the project. It covers the goals and non-goals, the architectural philosophy, the new five-layer mental model (Foundation, Sensors, Cognition, Effectors, Surfaces), every package, every app, every contract, the configuration system, the event bus, all canonical event flows, all five avatar modalities, the voice modes, the vision pipeline, the task router, testing and CI, the privacy posture, and the roadmap. It is intentionally exhaustive: it is meant to be a single document a new contributor can read once and then navigate the codebase with confidence.

1.1 At a glance

Attribute	Value
Project name	OpenMimicry
Version covered	1.0.0
License	MIT (all source + bundled character packs unless noted)
Languages	Python 3.11+ (backend, packages), TypeScript/React (frontend), Rust (Tauri shell), C# (Unity bridge sample)

Attribute	Value
Platforms	Windows 10/11, Linux (Ubuntu, Fedora), macOS (best-effort, secondary)
Top-level packages	8 (openmimicry-core, -llm, -voice, -stt, -tts, -avatar, -tasks, -vision)
Top-level apps	5 (backend, desktop/frontend, desktop/src-tauri, unity-bridge, external-echo)
Avatar rendering modalities	5 (Sprite2D, Three.js, Live3D, Unity, External)
LLM providers	Any LiteLLM-supported (OpenAI, Anthropic, Gemini, Bedrock, Azure, Ollama, OpenRouter, ...)
Task runtimes	Mock, LocalShell (allowlist), ClaudeCode (CLI), MCPAgent
Vision detectors	MediaPipe Hands, Pose, Face (off by default, consent-gated)
Tests	Unit + Contract + Integration + E2E + Frontend Vitest + Tauri Rust tests
CI/CD	GitHub Actions matrix (Ubuntu / Windows, Python 3.11 / 3.12)

1.2 Five things you can do with OpenMimicry

- Run a local LLM-backed companion that listens to you, replies in your voice of choice, and shows mood through an animated 2D sprite or a 3D avatar.
- Wake your companion by name, interrupt mid-sentence, or hold push-to-talk; all three modes coexist and are configurable on the fly.
- Delegate engineering tasks ("refactor this file", "run the tests") to Claude Code, an MCP-based agent, or an allowlisted local shell, then watch progress flow back into the overlay.
- Replace the avatar renderer end-to-end (Sprite2D, Three.js, Live3D, Unity, External) without touching the LLM, voice, vision, or task layers.
- Add a new gesture, a new LLM provider, a new task runtime, a new TTS engine, or a new avatar modality without editing the core: register a factory via Python entry points and the contract tests pick it up automatically.

2. Goals and non-goals

2.1 In scope

OpenMimicry is a local-first desktop companion framework. The framework is modular enough that any one of its capabilities — LLM, TTS, STT, vision, task runtime, avatar runtime, desktop shell — can be swapped without touching the others. The following are first-class goals:

- A small, typed, async-first Python core that owns state, events, and configuration.
- Clean adapter contracts for LLMs, STT, TTS, external task runtimes, avatar runtimes, and vision detectors/classifiers. Each contract is a Python Protocol or Pydantic schema; together they form the only cross-module surface.
- A pluggable avatar layer with five rendering modalities behind a single AvatarRuntimeAdapter Protocol: Sprite2D (default), Three.js (VRM + glTF), Live3D, Unity (via WebSocket), and External (generic protocol for third-party renderers).
- A Tauri 2 + React desktop shell with a transparent overlay window and a separate interactive panel window. Click-through is an OS-level toggle, not per-pixel hit testing.
- YAML-driven configuration, validated by Pydantic v2, layered with environment overrides, and selectable by profile.
- Reproducible installs, type-checked Python, Ruff-linted code, contract-tested adapters, GitHub Actions CI, and tagged semver releases.
- Optional vision (M13): MediaPipe Hands, Pose, and Face detection plus rule-based gesture and temporal movement classifiers, behind a strict off-by-default consent gate.
- A privacy posture that is the default, not an opt-in: nothing leaves the machine unless the user has explicitly chosen a cloud LLM or a cloud TTS engine, and any third-party adapter that talks to a cloud must log a startup warning naming the destination.

2.2 Explicitly out of scope (for now)

- Hosted, multi-user deployments. OpenMimicry is a local-first framework. A future plugin could publish state to a web service; that is a third-party concern.
- Mobile clients. The Tauri shell targets desktop platforms only.
- Training or fine-tuning models. OpenMimicry uses models; it does not train them.
- A plugin marketplace. Plugins are installed via standard Python entry points; a marketplace is a packaging concern for a possible future.
- Bundling Unity binaries inside the Python install. The Unity bridge is a protocol; a sample Unity project lives in apps/unity-bridge and is opened manually by the user.
- Cloud-hosted vision. The local-first posture is intentional; any third-party vision adapter that talks to a cloud must log a startup warning naming the destination.

-
- Face-mesh capture by default. Privacy-first: a face-mesh detector may ship as an opt-in plug-in but is not enabled out of the box.

3. Architectural philosophy

Four rules drive every decision in OpenMimicry. They are simple to state and ruthlessly applied; the rest of the document is an elaboration of them.

3.1 Hexagonal core

Domain logic lives behind small interfaces. Concrete libraries (LiteLLM, RealtimeTTS, RealtimeSTT, MediaPipe, mcp-agent, Tauri) sit on the outside and implement those interfaces. The avatar runtime never imports a concrete library. Tests import the mock; production imports the real thing; switching providers is a config change.

3.2 Events, not RPC

Modules talk to each other through a typed in-process event bus, not by reaching across module boundaries. The desktop UI subscribes to a stream of avatar directives; it does not pull on the LLM. A vision classifier publishes a `GestureDetected` event; no other module imports the classifier. Cross-module dataflow is by message-passing only.

3.3 One source of truth per concern

Configuration is loaded once into a validated `AppConfig`. Runtime state lives in a single `RuntimeStore`. Animation state lives in the avatar state machine. No module owns a copy of another module's state. When a value can live in two places it lives in zero — derive it on read instead.

3.4 Honest seams

Every adapter has an explicit contract, a runnable example implementation, a mock that satisfies the same contract, and a contract test that proves it. A new adapter is added by satisfying the contract, never by editing the core. This is the rule that makes "replace the LLM" or "add a new gesture" a one-package change instead of a refactor.

Why this matters. The four rules above are what make the architecture worth a portfolio. A graph of imports that respects them is shaped like a ring, not a hairball — every arrow points inward toward `openmimicry-core`, never sideways between siblings. CI enforces it with `scripts/check_imports.py`.

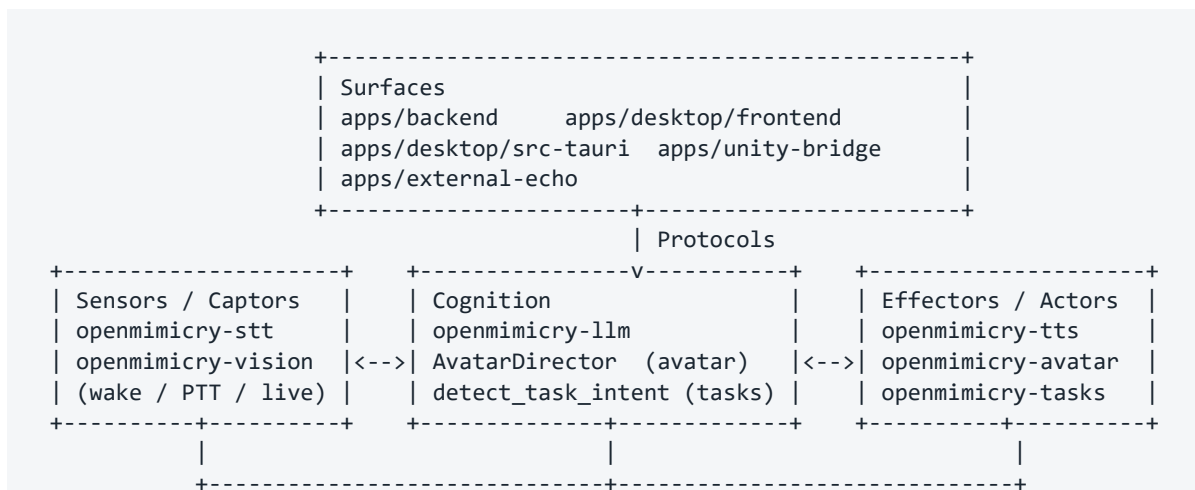
4. Layered architecture

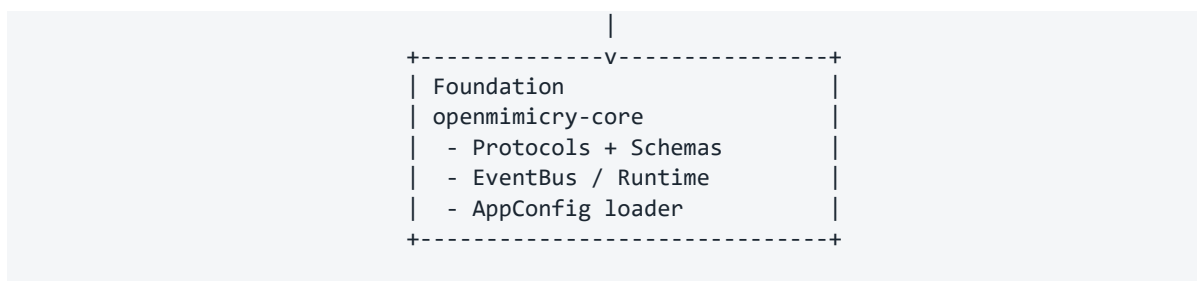
OpenMimicry's source code is split into five layers. Each layer talks to the next only through the frozen Protocols in `openmimicry-core` — never through sibling-package imports. That rule, enforced by `scripts/check_imports.py`, is what lets each layer be developed in isolation with mocks at every boundary.

4.1 The five layers

Layer	What it does	Packages / apps
Foundation	Frozen Protocols, schemas, EventBus, Runtime, AppConfig loader. Substrate every other layer is built on.	openmimicry-core
Sensors / Captors	Pure inputs. Observe the user or the world; publish RuntimeEvents.	openmimicry-stt (facade), openmimicry-vision
Cognition	Pure decision-making. Consume sensor events, decide what to do, emit directives.	openmimicry-llm + AvatarDirector + intent classifier
Effectors / Actors	Pure outputs. Act on the user or the world: speak, render, run shell tasks.	openmimicry-tts (facade), openmimicry-avatar, openmimicry-tasks
Surfaces	Presentation + assembly. UI, transports, the single file (wiring.py) where concrete adapters are imported.	apps/backend, apps/desktop/frontend, apps/desktop/src-tauri, apps/unity-bridge, apps/external-echo

4.2 Visual: the layered diagram





4.3 Rules every layer obeys

1. Contracts in, Protocols out. A layer's only legitimate import from outside its package is `openmimicry.core.*`. Sibling-package imports are forbidden (CI enforces it).
2. Mocks come first. Every layer ships a `MockWhatever` that satisfies its Protocol with zero optional dependencies. That mock is the canonical test fixture for the other layers.
3. Real implementations are optional installs. Heavy deps (LiteLLM, RealtimeSTT, MediaPipe, Tauri) live behind [extras] and are lazy-imported. Default pip install is mocks-only.
4. The assembly point is named. Exactly one file in the repository is allowed to import concrete adapter classes: `apps/backend/src/openmimicry_backend/wiring.py`. That is the surface where the layers are physically connected.

4.4 Conceptual layers vs installable packages

The layered mental model maps onto the physical packages as shown below. `openmimicry-voice` still exists as the source-of-truth implementation; `openmimicry-stt` and `openmimicry-tts` are thin facade packages that re-export the relevant half so each layer is also independently installable.

Layer	Packages	Notes
Foundation	<code>openmimicry-core</code>	Frozen contracts. Touch only via <code>docs/contracts.md</code> §11 change-control.
Sensors	<code>openmimicry-stt</code> , <code>openmimicry-vision</code>	<code>openmimicry-stt</code> is a facade over <code>openmimicry-voice</code> 's STT half.
Cognition	<code>openmimicry-llm</code> (+ <code>AvatarDirector</code> + intent classifier)	Cognition components live with their state — director in avatar, intent in tasks.
Effectors	<code>openmimicry-tts</code> , <code>openmimicry-avatar</code> , <code>openmimicry-tasks</code>	<code>openmimicry-tts</code> is a facade over <code>openmimicry-voice</code> 's TTS half.
Surfaces	<code>apps/backend</code> , <code>apps/desktop/frontend</code> , <code>apps/desktop/src-auri</code> , <code>apps/unity-bridge</code> , <code>apps/external-echo</code>	Only <code>wiring.py</code> imports concrete adapter classes.

5. Repository layout

The repository is a workspace: eight publishable Python packages and five deployable apps live side by side. Each Python package has its own `pyproject.toml`, its own version, and its own dependency set, but they install together from the workspace root.

```

openmimicry/
+-- packages/                                # 8 publishable Python packages
|   +-- openmimicry-core/                    # Foundation - frozen contracts + schemas + runtime
|   +-- openmimicry-llm/                     # Cognition - LiteLLM, mock, LLMRouter
|   +-- openmimicry-voice/                   # Source of truth for STT + TTS adapters
|   +-- openmimicry-stt/                     # Sensor-layer facade over openmimicry-voice (STT)
|   +-- openmimicry-tts/                     # Effector-layer facade over openmimicry-voice (TTS)
|   +-- openmimicry-avatar/                  # Effectors - five runtimes + AvatarDirector
|   +-- openmimicry-tasks/                   # Effectors - LocalShell, ClaudeCode, MCPAgent, intent
|   +-- openmimicry-vision/                  # Sensors - MediaPipe + classifiers (off by default)
+-- apps/
|   +-- backend/                             # FastAPI + WebSocket transport + wiring.py
|   +-- desktop/
|       |   +-- frontend/                    # React + Vite + Vitest
|       |   +-- src-tauri/                   # Tauri 2 shell (Rust)
|   +-- unity-bridge/                         # C# sample for the Unity adapter
|   +-- external-echo/                       # Reference WS echo server (External runtime)
+-- characters/                              # Bundled packs (sprite2d + vrm placeholder)
+-- config/
|   +-- profiles/                            # basic / voice / threejs / agent / vision / full ...
+-- docs/                                    # contracts.md + per-module briefs + architecture/
+-- scripts/                                # doctor.py, validate_pack, cleanup-legacy, win/
+-- tests/                                  # unit + contract + integration + e2e
+-- docker/                                 # Dockerfile.backend, Dockerfile.frontend-dev
+-- docker-compose.yml
+-- Makefile                                # Linux / macOS / WSL workflow (OS-aware)
+-- scripts/win/                            # Windows .bat wrappers around the Makefile targets
+-- pyproject.toml                          # Workspace root, uv-managed
+-- ROADMAP.md  CHANGELOG.md  README.md  CONTRIBUTING.md  SECURITY.md

```

5.1 Why this shape

- `packages/` uses a workspace (uv) so each package has its own `pyproject.toml` and version. They install together from the root for development; they could be published independently.
- `apps/backend/` is the only place that wires concrete adapters together. It is intentionally thin: business logic lives in `packages/`.
- `apps/desktop/` is split between the React app and the Rust/Tauri shell. They share no code with `packages/` other than over the wire (HTTP + WebSocket).
- `characters/` and `config/` ship as data. Packs are self-contained folders with a manifest (`pack.yaml`).

-
- Tests live at the repo root (tests/), not inside each package. That keeps cross-package integration tests first-class.
 - scripts/win/ provides .bat wrappers around every Makefile target so the developer experience is identical between Windows PowerShell and POSIX shells.

6. Foundation layer (openmimicry-core)

openmimicry-core is the substrate every other layer is built on. It contains nothing that talks to the network, the disk, or hardware. Only types, an event bus, and a runtime container.

6.1 What lives in core

- `openmimicry.core.contracts.*` — runtime-checkable Protocols for every cross-module boundary. Anything below extends or consumes these.
- `openmimicry.core.schemas.*` — frozen Pydantic v2 schemas (AvatarDirective, RuntimeEvent, LLMChunk, VisionFrame, Transcript, ...).
- `openmimicry.core.EventBus` — bounded-queue async pub/sub with per-subscriber backpressure.
- `openmimicry.core.Runtime` — wires the bus + config + logging into one async-context-manager that every other module receives at startup.
- `openmimicry.core.config.load` — YAML loader with env overlay, profile merge, and `schema_version` migrations.
- Structured logging tap (structlog) that mirrors every event to logs at DEBUG and significant ones at INFO.

6.2 What does NOT live in core

- Anything that opens a socket, a camera, or a microphone. Those are sensor/effector layer concerns.
- Concrete adapter classes. Those live in the implementation packages.
- HTTP / WebSocket transport. That is apps/backend's responsibility.

6.3 The Protocols

Eight cross-module Protocols live in openmimicry-core. Each is a Python Protocol decorated with `@runtime_checkable` so `isinstance()` works against any duck-typed implementation. None of them carry implementation; they are pure interfaces.

Protocol	File	Purpose
LLMAdapter	<code>contracts/llm.py</code>	<code>generate(messages, *, stream, tools) -> AsyncIterator[LLMChunk]</code>
STTAdapter	<code>contracts/voice.py</code>	<code>start(), stop(), transcripts: AsyncIterator[Transcript]</code>
TTSAdapter	<code>contracts/voice.py</code>	<code>speak(text stream, *, voice, on_chunk) -> Awaitable[None]; stop()</code>

Protocol	File	Purpose
SpeechController	contracts/voice.py	ptt_down/up, say(text), interrupt(), wake-mode toggles
WakeController	contracts/voice.py	enable(name), disable(); wake_events: AsyncIterator[WakeEvent]
TaskRuntimeAdapter	contracts/tasks.py	submit, status, cancel, updates stream; declares capabilities set
AvatarRuntimeAdapter	contracts/avatar.py	load_character, apply_directive, start/stop_speaking, healthcheck
AvatarDirector / Orchestrator	contracts/avatar.py	on_event(RuntimeEvent) -> Optional[AvatarDirective]; swap_runtime()
VisionAdapter + family	contracts/vision.py	LandmarkDetector, HandDetector, BodyDetector, HeadDetector, GestureClassifier, MovementClassifier

6.4 The schemas

Every cross-module value passes through a frozen Pydantic v2 model. Frozen means immutable: once constructed, the object cannot mutate. This eliminates an entire class of "who changed this field?" bugs and lets us safely fan out the same event to many subscribers.

Schema	Description
AppConfig	Top-level validated config tree (LLM, voice, avatar, tasks, vision, UI).
RuntimeEvent	Discriminated union covering UserTextSubmitted, UserSpeechStarted/Final, TranscriptPreview, LLMStarted/Token/Reply, TTSSStarted/Chunk/Finished/Interrupted, WakeDetected, TaskSubmitted/Updated/Completed, ConsentRequired, GestureDetected, MovementDetected, ConfigUpdated, Error.
AvatarDirective	Modality-independent: { state, emotion, animation?, speaking, text?, next_state?, duration_ms?, intensity?, gesture?, gaze?, metadata }.
State	idle listening thinking speaking happy error (extensible).
Emotion	neutral happy sad angry confused focused worried (extensible).
LLMChunk	{ delta, role, finish_reason?, usage? }.
Transcript	{ text, is_final, confidence?, segments? }.
TaskRequest / TaskUpdate / TaskResult	Task delegation lifecycle.

Schema	Description
TaskHandle / TaskStatus	
CharacterPack / EmotionFrames	Pack manifest and per-emotion animation config.
VisionFrame / HandLandmarks / BodyPose / HeadPose	Vision schemas.

6.5 The EventBus

A single in-process asyncio bus. Publish is non-blocking: each subscriber has its own bounded queue, and a slow subscriber cannot back-pressure a fast publisher into a deadlock. Two reasons it is worth doing in-process and not via Redis or similar:

- Everything runs in one backend process. Cross-process traffic only happens at the FastAPI / WebSocket boundary to the desktop.
- It keeps the test surface small: tests publish events and assert on what comes out.

The bus is the only path between modules. The desktop subscribes via WebSocket to a filtered projection (AvatarDirective, TranscriptPreview, LLMTokenStreamed, SpeechBubbleText, TaskUpdate, ConsentRequired, Error).

6.6 Runtime

`openmimicry.core.Runtime` is an async-context-manager that owns the EventBus, the AppConfig, the structured logger, and a small lifecycle registry. Every module receives the Runtime at startup and registers any lifespan hooks it needs (start, stop, healthcheck). When the runtime exits, hooks are shut down in reverse order.

6.7 The contract-freeze rule

The contract surface is frozen at v1.0. Any breaking change is treated as a major version bump. Additive amendments (new optional fields, new event variants) can land in minor versions but must follow the procedure in `docs/contracts.md` §11: PR updates the document, the matching code, the contract tests, and every consumer in lockstep.

7. Sensor layer

The sensor layer is everything that observes the user or the world. Pure inputs. Sensors publish `RuntimeEvents` on the bus; they never decide what the avatar should do — that is the cognition layer's job.

7.1 openmimicry-stt (facade)

`openmimicry-stt` is a facade package over the STT half of `openmimicry-voice`. The runtime implementation still lives in `openmimicry-voice`; the facade re-exports the relevant adapters under sensor-layer-appropriate names. Both imports work:

```
from openmimicry.voice import MockSTTAdapter    # still works
from openmimicry.stt import MockSTTAdapter     # preferred – layer-explicit
```

Both return the same class. The facade does not duplicate behaviour; it only renames it for the sensor-layer consumer.

Contents

Symbol	What it does
STTAdapter	Protocol every STT adapter must satisfy. Re-exported from <code>openmimicry.core.contracts.voice</code> .
MockSTTAdapter	Zero-dependency mock. Canonical fixture for the other layers' tests.
RealtimeSTTAdapter	RealtimeSTT-backed adapter. Heavy deps lazy-imported.
RealtimeSTTSettings, RealtimeSTTUnavailable	Pydantic settings + raised when RealtimeSTT isn't installed.
WakeController	Enable/disable wrapper for the live-wake flow.
SpeechController	Cross-layer controller that owns the single-TTS-task invariant; also re-exported for convenience.

How the STT runs

RealtimeSTT is configured for an assistant-style workflow: VAD (Silero by default), fast transcription, realtime text updates (`TranscriptPreview` events), wake words, and access to the raw audio stream. The live-wake flow runs STT continuously in wake-listening mode; when the wake name is detected, full transcription starts and the partial text streams to the speech bubble. Push-to-talk opens the mic on hotkey press and finalises on release.

7.2 openmimicry-vision (M13)

openmimicry-vision is the optional, off-by-default, consent-gated computer-vision sensor. It uses MediaPipe and OpenCV under the hood, exposes everything through `openmimicry.core.contracts.vision.*`, and is structured so a third party can add a new detector or classifier without editing the core.

Detectors

Detector	What it does
HandDetector (MediaPipe Hands)	21-point landmarks per hand, up to two hands.
BodyDetector (MediaPipe Pose)	33-point body landmarks including shoulders, elbows, wrists, hips, knees.
HeadDetector (MediaPipe Face)	Coarse head pose (yaw, pitch, roll) without face-mesh.

Classifiers

Two registries live alongside the detectors:

- Rule-based gesture classifier: `open_palm`, `fist`, `thumbs_up`, `point`, `peace`, `wave_pose`. Implemented with pure landmark geometry. Zero-training, deterministic, deterministic-test-friendly.
- Temporal movement classifier: `wave_motion`, `raised_hand`, `nodding`, `shaking_head`. Implemented as small sliding-window state machines over landmark streams.

Pipeline primitives

Vision ships three reusable primitives the rest of the framework can compose:

- VideoCapture — OpenCV worker thread that captures frames at a configured FPS and pushes them through a bounded queue. Single source of camera ownership; everything else subscribes.
- Throttle — drop frames so a downstream consumer never sees more than N per second.
- Debouncer — emit an event only after the same input has been stable for K frames. Eliminates flicker between gestures.

Privacy posture (mandatory)

5. Off by default. Vision's enabled: false schema default is non-negotiable. No other module turns it on automatically.

6. Consent on first activation. The adapter publishes `ConsentRequired` and refuses to open the camera until the frontend grants consent.
7. No upload, ever. Frames stay in-process. Any third-party classifier that ships later and does talk to a cloud must log a startup warning naming the destination — convention enforced by code review.

Entry-point registries

Vision's detectors and classifiers are registered through Python entry points, so a third party can ship a new model without editing the core:

```
[project.entry-points."openmimicry.contracts.vision_detector"]
movenet_pose = "yourpkg.movenet:make_movenet_pose_detector"

[project.entry-points."openmimicry.contracts.vision_gesture_classifier"]
rps          = "yourpkg.rps:make_rps_classifier"

[project.entry-points."openmimicry.contracts.vision_movement_classifier"]
side_swipe   = "yourpkg.side_swipe:make_side_swipe_classifier"
```

8. Cognition layer

Cognition is anything that decides. It consumes events from the sensor layer and produces directives that the effector layer acts on. Cognition is the only layer that does not map 1-to-1 with a single package — its components live wherever their state lives, so each component imports only from `openmimicry.core`.

8.1 Components

Component	Lives in	What it decides
LLMAdapter / LLMRouter	<code>openmimicry-llm</code>	Token-stream completions from any LiteLLM-compatible provider; the router chooses primary vs fallback.
AvatarDirector	<code>openmimicry-avatar</code> (<code>director.py</code>)	State machine that turns <code>RuntimeEvent</code> into <code>AvatarDirective</code> . The visible mood of the avatar.
<code>detect_task_intent</code>	<code>openmimicry-tasks</code> (<code>intent.py</code>)	Regex-first classifier that routes user text to LLM-vs-task.
SpeechController	<code>openmimicry-voice</code> (<code>controllers/speech.py</code>)	Coordinates STT + TTS — owns the single TTS task, barge-in policy, PTT cycle, live-wake projection.
TaskRouter	<code>openmimicry-tasks</code> (<code>router.py</code>)	Capability-based selection across registered <code>TaskRuntimeAdapters</code> .

8.2 `openmimicry-llm`

`openmimicry-llm` wraps LiteLLM. LiteLLM provides an OpenAI-style unified interface to many providers — OpenAI, Anthropic, Google Gemini, Bedrock, Azure, Ollama, OpenRouter, and more — so the avatar/runtime only ever has to know:

```
LLMAdapter.generate(messages, *, stream=True, tools=...)
```

And never:

```
OpenAIClient(...)
ClaudeClient(...)
OpenRouterClient(...)
OllamaClient(...)
GeminiClient(...)
```

Provider switching is a one-line config change (model: openrouter/anthropic/claude-3.5-sonnet -> model: ollama/llama3.1). The LLMRouter wraps an ordered list of LLMAdapters and retries on transient errors with exponential backoff; on terminal failure it falls back to the next adapter.

Adding a non-LiteLLM provider

LiteLLM already routes Ollama, OpenRouter, OpenAI, Anthropic, etc. Adding a provider it does NOT support is a three-step process:

8. Implement `openmimicry.core.contracts.llm.LLMAdapter` in a new module.
9. Register it via entry point: `[project.entry-points."openmimicry.contracts.llm"] yourpkg = "yourpkg.adapter:make_your_llm_adapter"`
10. The contract test parametrises over every registered factory — your implementation is exercised automatically.

8.3 AvatarDirector

The director is a small pure state machine. Given the current state and an incoming `RuntimeEvent`, it produces an optional `AvatarDirective`. The director never opens an I/O resource and never imports a runtime; it produces directives that the `AvatarOrchestrator` then dispatches to the chosen `AvatarRuntimeAdapter`.

Trigger event	Resulting directive
UserSpeechStarted	state=listening, speaking=false
LLMStarted (no tokens yet)	state=thinking, speaking=false
LLMTOKENStreamed + TTSStarted	state=speaking, speaking=true
TTSChunkSpoken boundary	speaking=true heartbeat
TTSFinished / TTSInterrupted	state=idle, speaking=false
Error	state=error, speaking=false
TaskCompleted (success)	state=happy, next_state=idle, duration_ms=cfg.celebration_ms
Task running, no TTS active	state=thinking, speaking=false

8.4 detect_task_intent

A regex-first classifier that decides whether a user utterance should go to the LLM or be routed as a task. Patterns are conservative and easily inspectable:

- "run the tests", "build the project", "open this in editor" — clear task verbs.

- "refactor X", "fix the bug in Y", "add a docstring to Z" — code-touching tasks.
- Anything else falls through to the LLM.

When intent matches a task pattern, a `TaskRequest` is constructed and sent to the `TaskRouter`. The classifier can be replaced or augmented by a small LLM-based classifier in the future — that is an additive amendment, not a contract change.

8.5 TaskRouter

The router is a capability-based dispatcher. Each `TaskRuntimeAdapter` declares a capabilities: `set[str]` (e.g. `{"shell", "safe"}` for `LocalShell`, `{"code", "refactor", "long-running"}` for `ClaudeCode`, `{"mcp", "filesystem"}` for `MCPAgent`). A `TaskRequest` carries `required: set[str]`; the router selects the first adapter whose capabilities are a superset. Ties are broken by config order.

9. Effector layer

Effectors are anything that acts on the user or the world. Pure outputs. Effectors consume directives and task requests from the cognition layer and produce: rendered avatar frames, spoken audio, shell output, agent task progress, and so on.

9.1 openmimicry-tts (facade)

Symmetric with openmimicry-stt: a thin facade over the TTS half of openmimicry-voice. The same code is reachable from openmimicry.voice or openmimicry.tts. The facade exists so that the effector layer is also a physical, independently installable package.

Contents

Symbol	What it does
TTSAdapter	Protocol every TTS adapter must satisfy.
MockTTSAdapter	Zero-dependency mock.
RealtimeTTSAdapter	RealtimeTTS-backed adapter; supports Coqui, Piper, Azure, OpenAI, and System TTS engines via the underlying library. Heavy deps lazy-imported.
RealtimeTTSSettings, RealtimeTTSUnavailable	Pydantic settings + raised when RealtimeTTS isn't installed.

Note that the `SpeechController` is deliberately NOT re-exported here. `SpeechController` coordinates STT and TTS together; it sits in the cognition layer (re-exported by openmimicry-stt for convenience).

9.2 openmimicry-avatar

The avatar package owns five rendering modalities, all behind the `AvatarRuntimeAdapter` Protocol. It also owns the `AvatarDirector` (cognition state machine — see chapter 8) and the `AvatarOrchestrator` (the small piece of glue that holds the current runtime and re-emits the latest directive when the runtime is swapped at runtime).

The five modalities

Modality	Renderer	Where it runs
Sprite2D	PNG frame sequences (per-emotion, with <code>_speaking</code> variants)	React frontend (canvas)
Three.js	VRM + glTF avatars via three.js	React frontend (WebGL)

Modality	Renderer	Where it runs
Live3D	Three.js + procedural mouth/idle/gaze drivers	React frontend (WebGL)
Unity	Standalone Unity build, drives an Animator over WebSocket	External Unity process
External	Any third-party renderer	External process speaking the public WS protocol

AvatarRuntimeAdapter (the contract)

```
class AvatarRuntimeAdapter(Protocol):
    async def load_character(self, pack: CharacterPack, cfg: dict) -> None: ...
    async def apply_directive(self, d: AvatarDirective) -> None: ...
    async def set_text(self, text: str | None) -> None: ...
    async def start_speaking(self, text: str | None = None) -> None: ...
    async def stop_speaking(self) -> None: ...
    async def set_visibility(self, visible: bool) -> None: ...
    async def healthcheck(self) -> bool: ...
    async def shutdown(self) -> None: ...
```

Adding a new modality means implementing this Protocol, registering a factory under the `openmimicry.contracts.avatar_runtime` entry-point group, shipping a mock, and adding the frontend component that paints the directive. The 17-step worked example for Three.js is documented in `docs/modules/M9_avatar_threejs.md`.

AvatarOrchestrator.swap_runtime invariant

When the user switches modality at runtime (e.g. Sprite2D -> Three.js), the orchestrator must re-emit the latest AvatarDirective after the swap so the new runtime arrives at the correct visible state immediately. Without this invariant, the avatar would freeze in idle until the next event arrives — broken UX.

9.3 openmimicry-tasks

`openmimicry-tasks` owns the TaskRuntimeAdapter Protocol, the TaskRouter, and three concrete adapters (plus a mock). Every task runtime declares capabilities and the router picks the right one for each TaskRequest.

Concrete adapters

Adapter	Capabilities	Notes
MockTaskAdapter	{mock}	Returns scripted progress; canonical test fixture.

Adapter	Capabilities	Notes
LocalShellAdapter	{shell, safe}	Strict allowlist of commands, shell=False, audit log, SIGTERM then SIGKILL on cancel.
ClaudeCodeAdapter	{code, refactor, long-running}	Invokes the Claude Code CLI in a subprocess, streams stdout.
MCPAgentAdapter	{mcp, filesystem, network}	Uses mcp-agent to build a per-task agent over a set of MCP servers.

Security posture for LocalShell

LocalShellAdapter is the security-sensitive reference implementation:

- Allowlist-only: every executable that may be run is named in the YAML allowlist. Anything else is rejected before a process is spawned.
- shell=False: subprocess is invoked with an argv array, never a shell string. No string interpolation in command construction.
- Audit log: every command, every argv array, every exit code, every cancel reason is written to ~/.openmimicry/audit.log with a timestamp.
- Cancel = SIGTERM, then SIGKILL after a configurable grace window.
- Mirror this posture for any runtime that touches the host filesystem.

10. Surface layer

Surfaces are how OpenMimicry is presented and assembled. They own the transports (HTTP, WebSocket), the UI shells (Tauri, browser, Unity), and — uniquely — the one file in the repository where concrete adapter classes are imported by name.

10.1 The wiring.py rule

Exactly one file in the repository is allowed to import concrete adapter classes:

```
apps/backend/src/openmimicry_backend/wiring.py
```

wiring.py reads AppConfig, picks the concrete class for each Protocol, instantiates it, and hands the dependency graph to the FastAPI app. Every other module — including the tests — receives its collaborators via constructor injection.

This is why a new contributor can grep for from openmimicry.llm.litellm and find every line that touches LiteLLM in the entire backend: one.

10.2 apps/backend

The backend is FastAPI on top of an asgi server (uvicorn by default). It owns four responsibilities:

- HTTP API: /chat (POST text), /mode (toggle wake/PTT/voice on the fly), /pack (swap character pack), /runtime (swap avatar modality), /admin (debug/dev endpoints), /config, /health.
- WebSocket endpoint: /ws projects a filtered subset of RuntimeEvents to the frontend (AvatarDirective, TranscriptPreview, LLMTokenStreamed, SpeechBubbleText, TaskUpdate, ConsentRequired, Error).
- Lifespan: starts the Runtime, the LLM router, the SpeechController, the AvatarOrchestrator, the TaskRouter, optionally the VisionAdapter, and shuts them down in reverse order on exit.
- Health: /health returns last-seen-OK timestamps per adapter and an overall ok flag.

10.3 apps/desktop/frontend

React + Vite + TypeScript. Vitest for unit tests. The frontend is intentionally a thin view:

- Connects to ws://localhost:8000/ws on mount.
- Receives AvatarDirective events and mounts one of five runtime components from src/runtimes/register.ts based on the directive's runtime_id.
- Sends user-input events to the backend via POST /chat or via Tauri-emitted PTT events.
- Does NOT own a state machine. The director on the backend is the source of truth.

Vite proxies HTTP and WS calls to the backend for browser-development mode: /chat, /health, /mode, /pack, /runtime, /admin, /config, /api, /ws, /static.

10.4 apps/desktop/src-tauri

Tauri 2 shell, written in Rust. It owns:

- Two windows: a transparent, always-on-top, click-through-by-default overlay window for the avatar; and a regular, interactive panel window for typing, settings, and conversation history.
- Global hotkey (PTT) registration. On hotkey down/up, emits ptt:down / ptt:up events into the webview.
- System-tray icon and menu (toggle interact mode, swap pack, swap runtime, quit).
- Bootstrap: starts the bundled backend (if configured) or assumes one is already running at the configured URL.

All Tauri ↔ frontend events use the namespace:event naming convention. Tauri 2.x rejects event names containing a dot, which earlier prototypes used; the convention is enforced by convention and by code review.

10.5 apps/unity-bridge

A sample Unity project (C#) that demonstrates how a Unity build can subscribe to the public WebSocket protocol and animate an Animator through AvatarDirective parameters. The Python side is UnityAvatarAdapter; the Unity side is a small C# script that opens a WebSocket and dispatches messages to the Animator. The Unity build is not bundled with the Python install — it is opened manually by a user who wants a Unity-driven avatar.

10.6 apps/external-echo

A reference implementation of the External runtime modality. It opens a WebSocket to the backend, prints every received directive to stdout, and demonstrates exactly what a third-party renderer (OBS plugin, web canvas, VR headset) would need to implement. It is also a useful CLI tool for debugging the avatar directive stream during development.

11. Event flows

Four canonical flows explain almost everything that happens at runtime. Each flow is a sequence of `RuntimeEvents` and `AvatarDirectives` across the bus. Full sequence diagrams live in `docs/event_flows.md`; the summaries below cover what every contributor should have in their head.

11.1 Text input

11. Frontend POSTs `/chat` with a text body.
12. Backend publishes `UserTextSubmitted` to the bus.
13. `AvatarDirector` emits `state=thinking`. Frontend repaints.
14. `LLMAdapter.generate` streams tokens. Per-chunk `LLMTokenStreamed` is published.
15. If `agent_voice` is enabled, `TTSAdapter.speak` consumes the stream; `TTSStarted`, `TTSChunkSpoken*`, `TTSFinished` follow.
16. `AvatarDirector` transitions `thinking -> speaking -> idle` as the events arrive.

11.2 Push-to-talk

17. User holds the configured hotkey. Tauri emits `ptt:down` to the webview, frontend POSTs to `/mode`.
18. `SpeechController.ptt_down()` opens the STT mic and stops any running TTS (interrupt).
19. `AvatarDirector` emits `state=listening`.
20. User releases the hotkey. STT finalises into `UserSpeechFinal`.
21. Same path as text from here on.

11.3 Wake-name live mode

22. `WakeController` runs the STT in wake-listening mode. STT publishes nothing visible while idle.
23. Wake name is heard. `WakeDetected` fires, full transcription starts.
24. Partial transcripts publish `TranscriptPreview` events for the speech bubble; the user sees their words appear live.
25. Final transcript -> `UserSpeechFinal` -> same path as text.
26. Barge-in: if `TTSStarted` is active while VAD detects voice, `SpeechController.interrupt()` stops TTS and the director transitions to listening.

11.4 Task delegation

27. Either an LLM tool-call or `detect_task_intent` produces a `TaskRequest`.
28. `TaskRouter` picks an adapter whose capabilities cover the request.

- 29. TaskRuntimeAdapter.submit returns a handle and an updates stream.
- 30. TaskSubmitted, TaskUpdated*, TaskCompleted events flow to the bus.
- 31. AvatarDirector reflects task state: thinking while running; happy on success; error on failure.

11.5 Vision gesture

- 32. Vision pipeline detects a stable gesture (e.g. open_palm) via the rule-based classifier.
- 33. GestureDetected event is published.
- 34. If a runtime-mapping is configured (e.g. "open_palm -> wave back"), AvatarDirector emits a one-shot directive with the matching emotion and animation.
- 35. If not mapped, the event is logged but no avatar change occurs.

12. Configuration system

Configuration is YAML, loaded once at startup, validated into an AppConfig Pydantic v2 model, and never read again as raw text. Three loader features matter:

- Profiles. config/profiles/*.yaml are layered onto config/app.yaml; switching profile is a one-flag change at startup (OPENMIMICRY_PROFILE=full-vision).
- Env overlays. Any field can be overridden by an environment variable using the double-underscore convention: OPENMIMICRY__LLM__MODEL=ollama/llama3.1.
- schema_version migrations. Breaking schema changes bump schema_version; the loader refuses to start on an older config without an explicit --migrate flag.

12.1 Profiles shipped at v1.0

Profile	What it enables
basic	Mocks-only. No LLM keys, no microphone, no camera. Great for first-run smoke.
voice	Real STT + TTS + Mock LLM. For voice testing without paying for tokens.
threejs	Three.js + VRM avatar runtime. Default Sprite2D swapped out.
agent	Adds ClaudeCode + LocalShell task runtimes. Capabilities routed through the TaskRouter.
vision	Enables the M13 vision pipeline (gesture + movement). Camera-on, consent-gated.
full	Everything: real LLM (via OPENROUTER_API_KEY), real voice, Three.js, agent tasks, vision.
full-vision	full + Vision profile combined; the kitchen sink.

12.2 Example app.yaml

```
schema_version: 1
app:
  log_level: INFO
  data_dir: ~/.openmimicry

llm:
  adapter: litellm
  model: openrouter/anthropic/claude-3.5-sonnet
  temperature: 0.7
  fallback:
    adapter: litellm
    model: ollama/llama3.1
```

```
voice:
  stt:
    adapter: realtimett
    language: en
    vad: silero
    wake:
      enabled: true
      names: ["Mimi", "Hey Mimi"]
  tts:
    adapter: realtimetts
    engine: coqui
    voice: en_female_1
    interruptible: true
  modes:
    text_always_on: true
    push_to_talk_hotkey: "Ctrl+Space"
    live_wake: true
    agent_voice: true

avatar:
  runtime: sprite2d
  pack: octomimic
  default_state: idle
  default_emotion: neutral
  transition_ms: 120

tasks:
  default_runtime: claude_code
  runtimes:
    claude_code: { adapter: claude_code, cli: claude }
    local_shell: { adapter: local_shell, allowlist: ["ls", "rg", "cat"] }
    mcp_agent: { adapter: mcp_agent, servers: [{name: filesystem, command: ["uvx", "mcp-server-filesystem", "~/projects"]}]}

vision:
  enabled: false           # default; opt-in only
  source: webcam
  fps: 15
  detectors: [hands, body]
  gestures: rules
  movements: temporal

ui:
  overlay: { width: 360, height: 360, click_through_default: true, always_on_top: true }
  panel: { width: 480, height: 720 }
```

12.3 The doctor script

scripts/doctor.py is a one-shot diagnostic that verifies Python version, Node version, Rust toolchain, ffmpeg, Tauri prerequisites, optional dependencies, config validity, and adapter reachability. It is the first thing a new contributor runs and the first thing CI runs on every PR.

13. Avatar modalities in depth

All five avatar runtimes implement the same AvatarRuntimeAdapter Protocol and are selected at runtime by the `avatar.runtime` config field. Switching modality at runtime triggers AvatarOrchestrator.swap_runtime, which re-emits the latest directive so the new runtime arrives at the right visible state.

13.1 Sprite2D (default)

PNG frame sequences laid out in per-emotion folders, with optional `_speaking` variants. The pack format is documented in docs/character_packs.md; the canonical example is the bundled octomimic pack.

Pack layout

```
characters/octomimic/
+-- pack.yaml
+-- idle/                # frame_0001.png, frame_0002.png, ...
+-- idle_speaking/
+-- listening/
+-- listening_speaking/
+-- thinking/
+-- thinking_speaking/
+-- speaking/
+-- happy/
+-- happy_speaking/
+-- error/
```

pack.yaml

```
schema_version: 1
id: octomimic
kind: sprite2d
default_state: idle
emotions:
  idle:      { frames: idle,      speaking_frames: idle_speaking,      fps: 5,  loop: true }
  listening: { frames: listening, speaking_frames: listening_speaking, fps: 8,  loop: true }
  thinking:  { frames: thinking,  speaking_frames: thinking_speaking, fps: 10, loop: true }
  speaking:  { frames: speaking,                                     fps: 12, loop:
true }
  happy:     { frames: happy,     speaking_frames: happy_speaking,      fps: 10, loop:
true, hold_ms: 1500, return_to: idle }
  error:     { frames: error,                                     fps: 6,  loop:
false }
```

13.2 Three.js (M9)

Renders VRM 1.0 and glTF 2.0 avatars in a Three.js scene mounted in the React overlay. Three.js, VRM loaders, and glTF loaders are lazy-loaded from CDN-equivalent npm packages.

- Directive -> animation clip mapping is declared per pack (idle, listening, thinking, speaking, happy, error).
- Camera, lighting, and background colour are config-driven.
- The same Three.js scene can be augmented with the Live3D drivers (chapter 13.3) without re-loading the model.

13.3 Live3D (M10)

Live3D layers procedural drivers on top of the Three.js modality:

- Mouth driver: amplitude-following blendshape animation from the TTS audio stream. Mouth opens proportionally to the spoken audio's RMS.
- Idle driver: small randomised breathing + head sway animation when speaking=false.
- Gaze driver: tracks user input position (mouse, or vision-derived head pose) and aligns the avatar's eye and head transforms toward it.
- Gesture driver: one-shot animation clips triggered by AvatarDirective.gesture (e.g. wave, point, thumbs_up).

13.4 Unity (M11)

UnityAvatarAdapter on the Python side opens a WebSocket and serialises AvatarDirectives as JSON. On the Unity side, a small C# script in apps/unity-bridge/ subscribes to the same WebSocket and dispatches messages to an Animator's parameters (state float, speaking bool, gesture trigger). The Unity build is launched as a separate process; the framework does not bundle Unity binaries.

13.5 External (M12)

The External modality is the explicit "any third-party renderer" path. The protocol is the same JSON-over-WebSocket dialect that Unity uses. apps/external-echo/ is the reference subscriber: it tails directives to stdout. Anyone can build an OBS plugin, a web canvas, a VR headset client, or a Stable-Diffusion frame painter on top of this.

14. Voice modes

Voice in OpenMimicry is three independent toggles that compose freely. Detail in docs/voice_modes.md; summary below.

14.1 Text-always-on

The text input is always available; pressing Enter submits a chat. This is the lowest-friction path and is the default behaviour.

14.2 Push-to-talk (PTT)

A globally-registered hotkey (default Ctrl+Space) opens the mic while held and closes it on release. PTT down also stops any running TTS, so the user can interrupt the avatar by holding the hotkey.

14.3 Live wake

STT runs in wake-listening mode (RealtimeSTT's built-in wake-word path). When the configured name (e.g. "Mimi", "Hey Mimi") is heard, full transcription starts. Partial transcripts populate the speech bubble while the user speaks.

14.4 Agent voice on/off

Independent of input mode, the avatar's spoken output (TTS) can be toggled at runtime via POST /mode (or from the tray). Useful when the user wants the LLM's text response but not the spoken one.

14.5 Barge-in (interruption)

If voice.tts.interruptible is true (default), and the STT VAD detects voice while TTS is playing, the SpeechController stops the active TTS task and transitions the avatar to listening. The current TTS task is cooperatively cancelled (RealtimeTTS exposes stop()); the LLM stream that fed it is also cancelled to avoid wasted tokens.

14.6 Echo handling

Audio loopback into the mic is mitigated by either (a) using a directional/USB mic, or (b) enabling RealtimeSTT's echo handling. We document both in the README under "Known issues" rather than silently doing the wrong thing.

15. Task delegation in depth

Task delegation is what lets the companion act, not just speak. Three concrete adapters ship at v1.0; the contract makes adding more a one-package change.

15.1 The contract

```
class TaskRuntimeAdapter(Protocol):
    capabilities: set[str]
    async def submit(self, req: TaskRequest) -> TaskHandle: ...
    async def status(self, handle: TaskHandle) -> TaskStatus: ...
    async def cancel(self, handle: TaskHandle) -> None: ...
    def updates(self, handle: TaskHandle) -> AsyncIterator[TaskUpdate]: ...
    async def result(self, handle: TaskHandle) -> TaskResult: ...
```

15.2 LocalShellAdapter — the safe baseline

- Allowlist-only. Every executable that may be invoked is named in YAML.
- shell=False. Commands are invoked with an argv array.
- Audit log. Every command is appended to ~/.openmimicry/audit.log with timestamp, argv, cwd, env-diff, exit code, and reason on cancel.
- Cancel = SIGTERM, then SIGKILL after a configurable grace period (default 3 s).
- POSIX-tested. Windows behaviour is best-effort and skipped in the live test on Windows.

15.3 ClaudeCodeAdapter

- Invokes the Claude Code CLI as a subprocess.
- Streams stdout into TaskUpdates.
- Capabilities: code, refactor, long-running. The router will pick this adapter for any TaskRequest carrying those capability requirements.
- Cancel forwards to SIGINT; if the CLI does not exit within the grace window, escalates to SIGTERM/SIGKILL.

15.4 MCPAgentAdapter

- Uses the mcp-agent library to construct a per-task agent backed by one or more MCP servers (filesystem, web, terminal, custom).
- Capabilities are derived from the configured MCP servers.
- Handles MCP connections, LLM workflows, external signals, and persistent state.

15.5 TaskRouter

Capability-based dispatch. A `TaskRequest` carries `required: set[str]`; the router selects the first adapter (in config order) whose capabilities is a superset of required. If none match, the router publishes an Error event and refuses the request.

16. Vision (M13) in depth

Vision is OpenMimicry's optional sixth modality. It uses MediaPipe under the hood, exposes everything through `openmimicry.core.contracts.vision.*`, and is off by default. The next chapter covers the privacy posture in detail; this chapter is the technical brief.

16.1 Pipeline

Frames are captured by a single VideoCapture worker thread; everything else subscribes to its bounded queue. The pipeline is configured to fan out to multiple detectors per frame so a single capture can drive multiple downstream classifiers.

```
WebcamFrame -> VideoCapture (worker thread, bounded queue)
               -> Throttle (drop to N FPS, default 15)
               -> HandDetector   -> Debouncer   -> GestureClassifier -> GestureDetected
               -> BodyDetector   -> Debouncer   -> MovementClassifier -> MovementDetected
               -> HeadDetector  -> coarse yaw/pitch/roll           -> HeadPoseUpdated
```

16.2 Gesture rules (zero-training)

Gesture rules are implemented as pure landmark geometry. Examples:

- `open_palm`: all five fingertip-to-wrist distances exceed a threshold and all fingers are extended (tip $y < \text{pip } y$ for vertical hands).
- `fist`: all five fingertip-to-wrist distances are below a threshold.
- `thumbs_up`: thumb tip is above thumb mcp, other fingers are folded, hand is roughly vertical.
- `point`: index finger extended, other fingers folded.
- `peace`: index and middle extended, others folded.

16.3 Movement rules (temporal)

Movement classifiers are small state machines over sliding windows of landmarks:

- `wave_motion`: wrist's x oscillates across the screen-centre line at least N times within W seconds.
- `raised_hand`: wrist y stays above shoulder y for at least K consecutive frames.
- `nodding`: head pitch oscillates around neutral with amplitude $> T$ within W .
- `shaking_head`: head yaw oscillates around neutral with amplitude $> T$ within W .

16.4 Plug-in registries

Three Python entry-point groups make vision third-party-extensible:

```
openmimicry.contracts.vision_detector  
openmimicry.contracts.vision_gesture_classifier  
openmimicry.contracts.vision_movement_classifier
```

A third party can ship a new detector (e.g. MoveNet, BlazePose), a new gesture classifier (e.g. rock-paper-scissors), or a new movement classifier (e.g. side-swipe) without editing the core. The contract tests parametrise over every registered factory and exercise each implementation automatically.

16.5 Why MediaPipe

MediaPipe was chosen for v1.0 because it has:

- Permissively licensed pre-trained landmark models for hands, body, and face.
- CPU inference fast enough for 15 FPS on a mid-range laptop.
- A Python wheel that ships its own runtime, so no extra setup is required beyond pip install.
- A stable landmark schema (21 hand points, 33 body points) that maps cleanly to gesture rules.

Holistic (M16 on the roadmap) would merge hands + body + face in one pass; the contracts already accommodate it.

17. Privacy and security posture

OpenMimicry is built local-first. This chapter is the user-facing summary of every privacy and security decision in the framework.

17.1 What stays on the machine by default

- Microphone audio. STT runs in-process. No upload.
- Webcam frames. The vision module is off by default and requires explicit user consent to start the camera. Frames stay in-process.
- Configuration and conversation history. `~/.openmimicry` is local.

17.2 What does leave the machine — only when configured

- LLM completions, if a cloud LLM provider is configured (e.g. OpenRouter, Anthropic). Switching to `ollama/llama3.1` keeps inference local.
- TTS audio, if a cloud TTS engine is configured (e.g. Azure, OpenAI TTS). Coqui and Piper run locally.
- Anything an MCP server is configured to talk to (e.g. an HTTP MCP server).

17.3 Mandatory consent gate for vision

The vision module follows a strict three-rule posture:

36. Off by default. Vision's enabled: `false` is non-negotiable.
37. Consent on first activation. The adapter publishes `ConsentRequired` and refuses to open the camera until the frontend grants consent.
38. No upload, ever. Frames stay in-process. Any third-party classifier that ships later and does talk to a cloud must log a startup warning naming the destination — convention enforced by code review.

17.4 LocalShellAdapter safety

LocalShellAdapter is the security-sensitive reference task runtime:

- Allowlist-only — anything not in the YAML allowlist is rejected before a process is spawned.
- `shell=False` — subprocess uses an `argv` array, never a shell string.
- Audit log — every command, `argv`, exit code, and cancel reason is appended to `~/.openmimicry/audit.log`.
- Cancel = `SIGTERM` then `SIGKILL` after a grace window.
- Mirror this posture for any runtime that touches the host filesystem.

17.5 Disclosed third-party calls

LiteLLM ships its own list of supported providers. Any of them, when configured, will be called from the host. This is documented in the README under "Network egress". A startup log entry names the destination.

17.6 Secrets management

Secrets (API keys) are loaded from `.env` (recommended) or from environment variables. `.env` files are `.gitignored`. The shipped `.env.example` documents every supported key and gives PowerShell and Bash loading snippets.

18. Testing, CI, and release

18.1 The four test layers

Layer	Location	What it asserts
Unit	tests/unit/<package>/ and inside each package	Pure functions, schemas, state machines, single-class adapters with mocks.
Contract	tests/contract/	Every registered factory satisfies the Protocol shape. Parametrised over entry-point discovery.
Integration	tests/integration/	EventBus + mock adapters wired together; full event flows asserted end-to-end.
E2E smoke	tests/e2e/	FastAPI test client + WS projection. Optional Playwright pass against the panel route.

18.2 Coverage

pytest-cov runs on every PR. v1.0 ships with coverage gates per package, the strictest being on openmimicry-core ($\geq 95\%$, line + branch). Other packages target $\geq 85\%$; e2e is not coverage-gated.

18.3 Linting and typing

- Ruff for linting and formatting (ruff check, ruff format --check). One config (pyproject.toml) for the whole workspace.
- Pyright for type-checking. Strict on openmimicry-core; gradual elsewhere with a per-package floor.
- scripts/check_imports.py enforces the layered-architecture rule: packages may only import from openmimicry.core.*, never sibling packages.

18.4 Frontend tests

- Vitest for unit tests of the runtime registry, the message handlers, the directive-mapping code, and the runtime components (with vi.mock for three.js, etc.).
- Playwright (opt-in) for an end-to-end smoke against the running stack.

18.5 Tauri tests

apps/desktop/src-tauri/tests/ contains Rust integration tests for the shell-level concerns: tray menu construction, overlay state transitions, hotkey registration, and the cross-window event-naming convention.

18.6 GitHub Actions workflows

- `ci.yml` — on PR. Ruff + Pyright + pytest (unit + contract + integration) on Ubuntu and Windows, Python 3.11 and 3.12. Vitest on Node 22. Tauri tests on Linux.
- `e2e.yml` — on main and tags. Brings up the backend, runs the e2e smoke + Playwright.
- `release.yml` — on tag `vX.Y.Z`. Builds Python wheels, Tauri bundles for Windows and Linux, generates a changelog from Conventional Commits, publishes a GitHub Release.

18.7 Versioning

SemVer. Within 1.x: patch (1.0.1) for bug fixes and additive optional fields; minor (1.1.0) for new optional modules or extras; major (2.0.0) only when a Protocol field actually has to change. A major bump always ships with a migration note and a `schema_version` increment in the same PR.

19. Engineering practices

The project is meant to be portfolio-grade, not a prototype. The non-functional requirements below are part of the deliverable; they are not optional.

19.1 Tooling baseline

Tool	Role
Ruff	Linting + formatting
Pyright	Type checking
Pytest + pytest-asyncio + pytest-cov	Testing
uv	Python dependency management (workspace-aware)
pnpm 11	Frontend dependency management
Vite + Vitest	Frontend bundler + tests
Tauri 2	Desktop shell (Rust)
GitHub Actions	CI/CD
Make + .bat wrappers	Cross-platform developer commands
Docker + docker-compose	Optional containerised dev (multi-stage builds)

19.2 Conventional commits and changelog

Commit messages follow Conventional Commits (feat:, fix:, docs:, ci:, refactor:, test:, chore:, etc.). The release workflow generates CHANGELOG.md sections from the commit log on each tag.

19.3 Pre-commit hooks (optional)

A pre-commit config covers Ruff, end-of-file fixers, large-file refusal, and a fast-path check_imports invocation. Pre-commit is recommended but not enforced; CI is the source of truth.

19.4 Contributor documents

- CONTRIBUTING.md — how to set up a dev environment, the layered-architecture rules, the test-running cookbook, the PR checklist.
- CODE_OF_CONDUCT.md — Contributor Covenant 2.1.
- SECURITY.md — vulnerability disclosure procedure.

- MAINTAINERS.md — list of current maintainers.
- ROADMAP.md — shipped milestones + post-v1.0 ideas (M14-M20).

19.5 Docker

docker-compose.yml runs the backend (Python 3.12-slim multi-stage build) and, under the dev profile, the frontend (pnpm + Node 22). Tauri is deliberately not dockerised: native desktop windows do not play well with containers and the docker path is for backend-only headless work (e.g. CI smoke against the WS protocol).

20. Running from zero

This chapter is a condensed install + first-run walkthrough. Full step-by-step lives in README.md.

20.1 Prerequisites

Tool	Minimum version	Why
Python	3.11+	Runs the backend and packages.
Node.js	22 LTS	Frontend bundler + Tauri prerequisites.
pnpm	11	Frontend package manager.
Rust + Tauri prerequisites	stable + WebView2 / WebKitGTK	Builds the desktop shell.
ffmpeg	any recent	Used by RealtimeTTS / RealtimeSTT engines.

20.2 Three workflows

Browser dev (no Tauri)

Two terminals:

```
# Terminal 1
make backend          # Linux/macOS/WSL
scripts\win\backend.bat # Windows

# Terminal 2
make frontend
scripts\win\frontend.bat
```

Open <http://localhost:5173>. The Vite dev server proxies HTTP and WS calls to the backend on :8000.

Native Tauri (recommended for everyday use)

Two terminals:

```
# Terminal 1
make backend
scripts\win\backend.bat

# Terminal 2 - Tauri's beforeDevCommand starts its own Vite
make desktop
scripts\win\desktop.bat
```

Do NOT also run make frontend in this workflow — both would try to bind :5173.

Docker (backend only)

```
make docker-build
make docker-up
# stops with:
make docker-down
```

20.3 Configuring an LLM

Copy .env.example to .env. Set OPENROUTER_API_KEY (or ANTHROPIC_API_KEY, OPENAI_API_KEY, etc. — whichever provider you choose). The .env file is .gitignored. PowerShell snippets are documented inside .env.example.

Or skip cloud entirely: install Ollama, pull a model (ollama pull llama3.1), set OPENMIMICRY_PROFILE=basic, change llm.model to ollama/llama3.1.

21. Milestones (M0 - M13)

The framework was built as a sequence of numbered milestones. Each ships behind a frozen contract; the same adapter Protocol covers every implementation past and future.

21.1 Phase 0 — Contract freeze

openmimicry-core landed first. Every Protocol, every schema, every event variant, and the §9 wire protocol stay stable for the entire 1.x line. No other module could start until this was frozen. This is the rule that makes everything else movable.

21.2 MX — Tooling baseline

Ruff, Pyright, Pytest, GitHub Actions skeleton, pnpm hardening, the import-checker, the doctor script. Done early so every subsequent milestone benefits from the safety net.

21.3 M0 — Core runtime

EventBus, Runtime, AppConfig loader, structlog, schema_version migration scaffolding.

21.4 M1 — openmimicry-llm

MockLLMAdapter, LiteLLMAdapter, LLMRouter with retry-and-fallback. The router wraps an ordered list of LLMAdapters and falls back on terminal errors with exponential backoff.

21.5 M2 — openmimicry-voice

MockSTTAdapter, MockTTSTAdapter, RealtimeSTTAdapter, RealtimeTTSTAdapter, SpeechController, WakeController. The single-TTS-task invariant is enforced inside SpeechController.

21.6 M3 — openmimicry-avatar core

Pack loader, AvatarDirector, AvatarOrchestrator. No runtime yet — the runtime ships in M4.

21.7 M4 — Sprite2D

Sprite2DAvatarAdapter (Python) + React runtime component. The bundled octomimic pack is the reference. PNG frames, per-emotion folders, optional _speaking variants.

21.8 M5 — openmimicry-tasks

TaskRouter, MockTaskAdapter, LocalShellAdapter (allowlist + audit), ClaudeCodeAdapter, MCPAgentAdapter, detect_task_intent regex classifier.

21.9 M6 — apps/backend

FastAPI app, wiring.py, WS projection, HTTP API, /health, lifespan management.

21.10 M7 — apps/desktop/frontend

React + Vite + TypeScript + Vitest. Speech bubble, voice controls, mode toggles, runtime registry, panel route.

21.11 M8 — apps/desktop/src-tauri

Tauri 2 shell. Two windows (transparent overlay + interactive panel), tray menu, global PTT hotkey, namespace:event naming convention, .pdb-collision fix on Windows.

21.12 M9 — Three.js modality

ThreeJSAvatarAdapter (Python) + React Three.js runtime. VRM 1.0 + glTF 2.0. The canonical 17-step add-a-modality template lives in docs/modules/M9_avatar_threejs.md.

21.13 M10 — Live3D

Mouth driver (TTS amplitude), idle driver (breathing + sway), gaze driver (input position), gesture driver (one-shot clips). All layered on top of M9 without re-loading the model.

21.14 M11 — Unity bridge

UnityAvatarAdapter on the Python side. apps/unity-bridge/ contains a small Unity sample project with a C# script that opens a WS and dispatches messages to an Animator.

21.15 M12 — External

ExternalAvatarAdapter + the apps/external-echo reference subscriber. Same JSON-over-WS protocol Unity speaks; opens the door for OBS plugins, web canvases, VR headsets, anything.

21.16 M13 — openmimicry-vision

MediaPipe Hands, Pose, Face. Rule-based gesture + temporal movement classifiers. Plug-in registries via Python entry points. Consent-gated, off by default.

22. Roadmap (post-v1.0)

v1.0 is the contract-freeze release. Post-v1.0 ideas extend the same M-series numbering so contract-amendment PRs reference matching numbers.

Milestone	Idea
M14	Body-pose gesture classifier on top of BodyPose. M13 ships the schema + detector; the classification rules are the missing piece.
M15	Live2D modality. Plug-in Live2D Cubism rendering inside the existing Three.js scene mount.
M16	Holistic detector. MediaPipe Holistic (hands + body + face in one pass) as a single LandmarkDetector.
M17	Cloud LLM provider extras. Per-provider extras for Anthropic, Google, OpenAI, Bedrock, Azure beyond the generic LiteLLM path.
M18	Plugin packaging. A cookiecutter template that scaffolds a third-party plugin (custom avatar runtime + classifier + LLM provider) wired through entry points.
M19	Frontend consent UX. The vision-consent dialog + indicator dot. (M13 ships the bus event; M7 owns the UI.)
M20	Live demo site. Static site that runs the frontend against a hosted demo backend (mocks only).

22.1 Cadence

- Patch (1.0.1, ...) — bug fixes, doc edits, additive optional fields.
- Minor (1.1.0, ...) — new optional modules, adapters, extras. Contracts gain additive amendments only.
- Major (2.0.0) — when (if) a Protocol field actually has to change. Migration note + schema_version bump in the same PR.

23. FAQ

23.1 Why is it called OpenMimicry?

Mimicry: the framework mimics a companion — it listens, it speaks, it watches, it acts. Open: every concrete adapter is replaceable, the source is MIT, and the contracts are public. The name is also a quiet nod to the cephalopod-themed bundled character (Octomimic) and the broader idea that the avatar is a surface, not the agent — what is doing the listening and replying lives behind the contracts.

23.2 Why both openmimicry-voice and openmimicry-stt/-tts?

openmimicry-voice is the source-of-truth implementation. openmimicry-stt and openmimicry-tts are facade packages that re-export the relevant half. The split exists so the layered mental model is also a physical, independently installable model: a sensor-only contributor can pip install openmimicry-stt without dragging in TTS dependencies, and an effector-only contributor can pip install openmimicry-tts without dragging in STT dependencies.

23.3 Can I use OpenMimicry without an LLM key?

Yes. Install Ollama, pull a small model, point `llm.model` at `ollama/llama3.1`. Or use `OPENMIMICRY_PROFILE=basic` which wires the `MockLLMAdapter` — that returns scripted text and is perfect for first-run smoke testing and CI.

23.4 Why Tauri instead of Electron?

- Smaller binary. Tauri bundles the OS's webview, not a full Chromium.
- Better OS integration. Click-through, always-on-top, transparent windows, tray, and global hotkeys are first-class in Tauri.
- Rust core. The Tauri shell is small and easy to keep audit-able.

23.5 Why MediaPipe instead of TensorFlow.js or PyTorch?

Best ratio of CPU performance to setup pain for v1.0. MediaPipe wheels ship their own runtime; no extra installs. The contracts allow swapping detectors (MoveNet, BlazePose, Holistic) without touching anything outside openmimicry-vision.

23.6 Why an event bus and not just function calls?

Three reasons: (1) cross-module dataflow stays one-directional and inspectable — every cross-module value crosses the bus; (2) tests can publish events and assert on the resulting projection

without standing up the whole stack; (3) adding a new subscriber (e.g. an analytics tap, a memory layer, a remote debugger) does not require modifying existing producers.

23.7 Why one wiring.py and not dependency injection magic?

Because explicit is better than magic. wiring.py is fifty lines you can read. It reads AppConfig, instantiates concrete classes, hands them to the FastAPI app. Anyone tracing a bug can grep the file once and know exactly what is connected to what. DI frameworks that scan annotations make this a multi-file mystery.

24. Glossary

Term	Meaning
Adapter	Concrete class implementing one of openmimicry-core's Protocols (e.g. RealtimeSTTAdapter implements STTAdapter).
AvatarDirective	Frozen Pydantic schema describing the avatar's intended visible state. Modality-independent.
AvatarOrchestrator	Holds the current AvatarRuntimeAdapter; dispatches directives; re-emits on swap.
AvatarRuntimeAdapter	Protocol every avatar modality implements (Sprite2D, Three.js, Live3D, Unity, External).
Barge-in	TTS interruption caused by VAD detecting user voice during playback.
CharacterPack	Self-contained folder with a pack.yaml manifest and the assets it references (sprites, VRM, gestures, ...).
Contract	A frozen Pydantic schema or a runtime-checkable Python Protocol declared in openmimicry-core.
EventBus	In-process async pub/sub. Single bus per Runtime.
Facade package	Package that re-exports another package's symbols under a new name (openmimicry-stt, openmimicry-tts).
LiteLLM	Library that gives many LLM providers a unified OpenAI-style interface.
MCP	Model Context Protocol. mcp-agent uses it to build agents over MCP-server-exposed capabilities.
MockAdapter	Zero-dependency reference implementation of a Protocol; canonical test fixture.
Profile	YAML overlay file in config/profiles/ that selects which adapters and features are enabled.
PTT	Push-to-talk. Hotkey-held mic capture.
RealtimeSTT / RealtimeTTS	Libraries providing low-latency speech-to-text / text-to-speech with VAD, wake-word support, and engine fallback.
RuntimeEvent	Discriminated union of every event variant the EventBus carries.
SpeechController	Cross-layer controller that owns the single-active-TTS-task invariant and the barge-in policy.
TaskRouter	Capability-based dispatcher; selects the right TaskRuntimeAdapter for each TaskRequest.
VAD	Voice activity detection. Silero VAD by default.

Term	Meaning
WakeController	Thin enable/disable wrapper for the live-wake mode.
wiring.py	The single file in the repo that imports concrete adapter classes by name.

25. Appendix A — Make targets and Windows wrappers

Make target	Windows wrapper	What it does
make install	scripts\win\install.bat	uv pip install -e all packages + pnpm install + sanity checks
make doctor	scripts\win\doctor.bat	Diagnose Python/Node/Rust/Tauri/ffmpeg + config + adapter reachability
make backend	scripts\win\backend.bat	Run apps/backend in dev mode (uvicorn --reload)
make backend-prod	—	Run apps/backend without --reload
make frontend	scripts\win\frontend.bat	vite dev (browser workflow only)
make frontend-build	—	vite build
make frontend-test	—	vitest run
make frontend-typecheck	—	tsc --noEmit
make desktop	scripts\win\desktop.bat	tauri dev (Tauri's beforeDevCommand starts its own Vite)
make desktop-build	—	tauri build (Windows .msi/.exe, Linux .deb/.AppImage)
make desktop-test	—	cargo test in src-tauri
make test	scripts\win\test.bat	pytest unit + contract + integration
make docker-build	—	docker-compose build
make docker-up	scripts\win\docker-up.bat	docker-compose up (backend + optional frontend-dev)
make docker-down	—	docker-compose down
make cleanup-legacy	scripts\win\cleanup-legacy.bat	Purge legacy folders (dry-run by default; --apply to commit)
make release-preview	—	Build wheels + bundles into ./dist for manual inspection

26. Appendix B — Entry-point groups

Plugins extend OpenMimicry through standard Python entry points. The contract tests discover registered factories and exercise each one against its Protocol. Sample pyproject.toml stanza:

```
[project.entry-points."openmimicry.contracts.llm"]
mock      = "openmimicry.llm.mocks:make_mock_llm"
litellm   = "openmimicry.llm.litellm_adapter:make_litellm_adapter"

[project.entry-points."openmimicry.contracts.stt"]
mock      = "openmimicry.voice.mocks:make_mock_stt_adapter"
realtimestt = "openmimicry.voice.stt.realtimestt_adapter:make_realtimestt_adapter"

[project.entry-points."openmimicry.contracts.tts"]
mock      = "openmimicry.voice.mocks:make_mock_tts_adapter"
realtimetts = "openmimicry.voice.tts.realtimetts_adapter:make_realtimetts_adapter"

[project.entry-points."openmimicry.contracts.task_runtime"]
mock      = "openmimicry.tasks.mocks:make_mock_task_adapter"
local_shell = "openmimicry.tasks.local_shell_adapter:make_local_shell_adapter"
claude_code = "openmimicry.tasks.claude_code_adapter:make_claude_code_adapter"
mcp_agent  = "openmimicry.tasks.mcp_agent_adapter:make_mcp_agent_adapter"

[project.entry-points."openmimicry.contracts.avatar_runtime"]
sprite2d = "openmimicry.avatar.runtimes.sprite2d:make_sprite2d_adapter"
threejs  = "openmimicry.avatar.runtimes.threejs:make_threejs_adapter"
live3d   = "openmimicry.avatar.runtimes.live3d:make_live3d_adapter"
unity    = "openmimicry.avatar.runtimes.unity:make_unity_adapter"
external = "openmimicry.avatar.runtimes.external:make_external_adapter"

[project.entry-points."openmimicry.contracts.vision_detector"]
mediapipe_hands = "openmimicry.vision.detectors.mp_hands:make_mp_hands_detector"
mediapipe_pose  = "openmimicry.vision.detectors.mp_pose:make_mp_pose_detector"
mediapipe_face  = "openmimicry.vision.detectors.mp_face:make_mp_face_detector"

[project.entry-points."openmimicry.contracts.vision_gesture_classifier"]
rules = "openmimicry.vision.classifiers.gestures:make_rule_gesture_classifier"

[project.entry-points."openmimicry.contracts.vision_movement_classifier"]
temporal = "openmimicry.vision.classifiers.movements:make_temporal_movement_classifier"
```

27. Closing notes

OpenMimicry's design goal was always simple: build a desktop companion that does not lock you into one model, one voice, one renderer, or one task runner. The substance of that goal is the contract surface in `openmimicry-core`. Every other word in this document is downstream of that surface.

If you take only one thing from this document, take this: the system is shaped like a ring. Every arrow points inward toward `openmimicry-core`. There are no sibling-to-sibling imports. There is one assembly point (`apps/backend/src/openmimicry_backend/wiring.py`). Everything else is replaceable, mockable, and testable in isolation. That is what makes the framework portfolio-worthy and what will make it survive contributors who have never met.

Welcome to OpenMimicry. Enjoy the build.

— end of design document —